Deep Learning

Lecture 4: Training Models

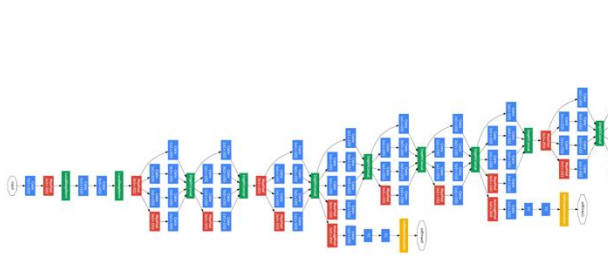
Amir Atapour-Abarghouei

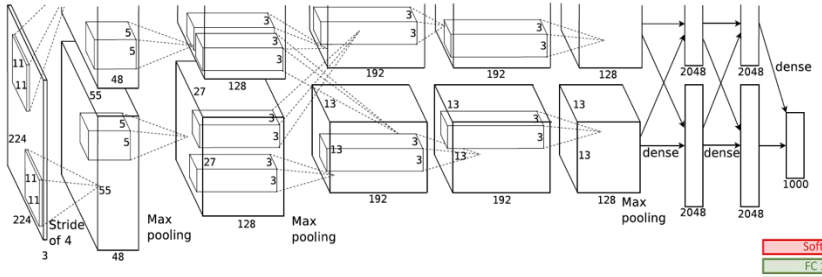
amir.atapour-abarghouei@durham.ac.uk

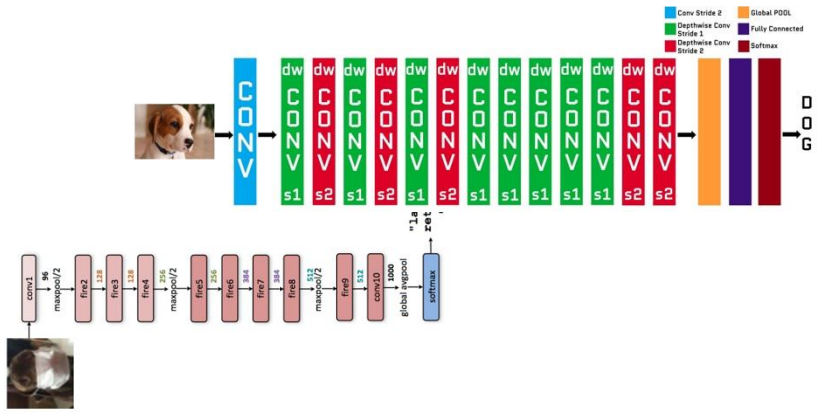
Durham University

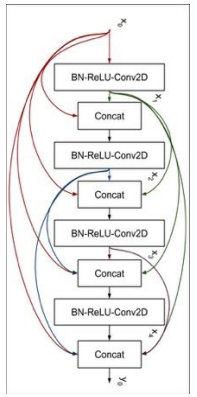
Lecture Overview

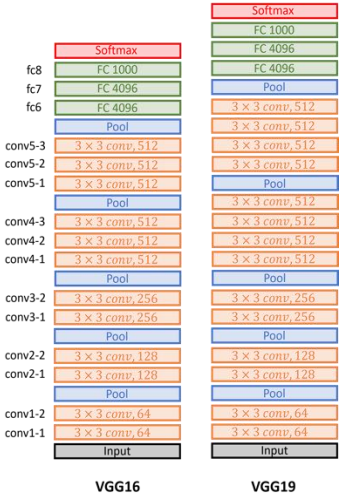
1. Convolutional Architectures and Backbones
2. Building Blocks of Neural Networks
3. Improved Training











ImageNet

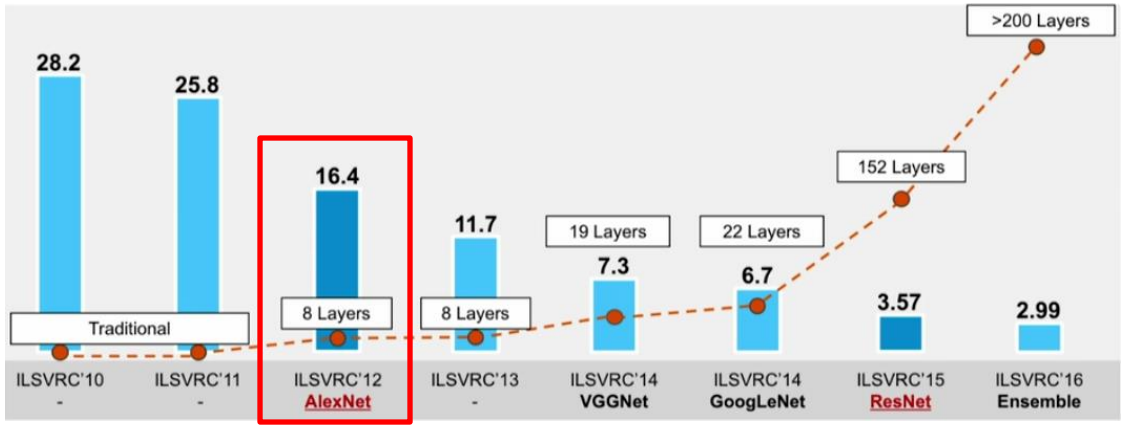
Deep Models and Big Data:

- ImageNet Large Scale Visual Recognition Challenge.
  - More than 14 million images for different tasks.

http://www.image-net.org/

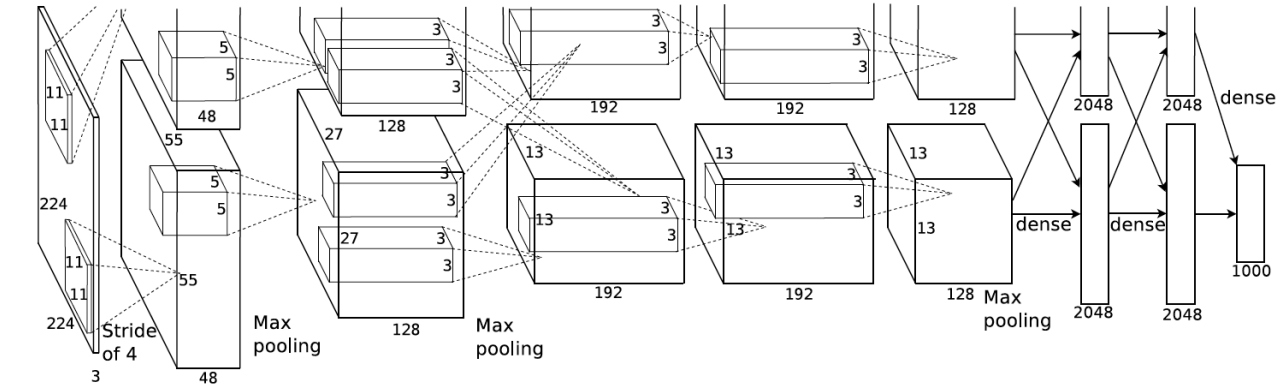
Architecture Design

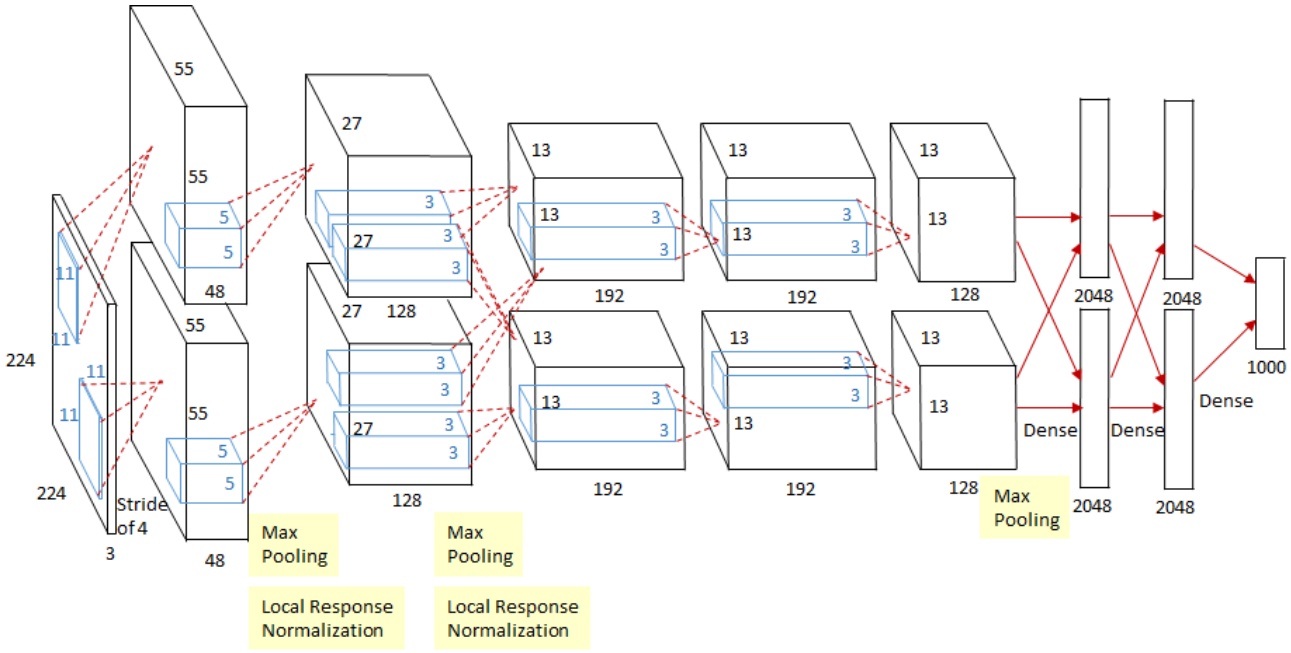
AlexNet [Krizhevsky et. al, 2012]



First CNN-based winner of the ImageNet Large Scale Visual Recognition Challenge in 2012.

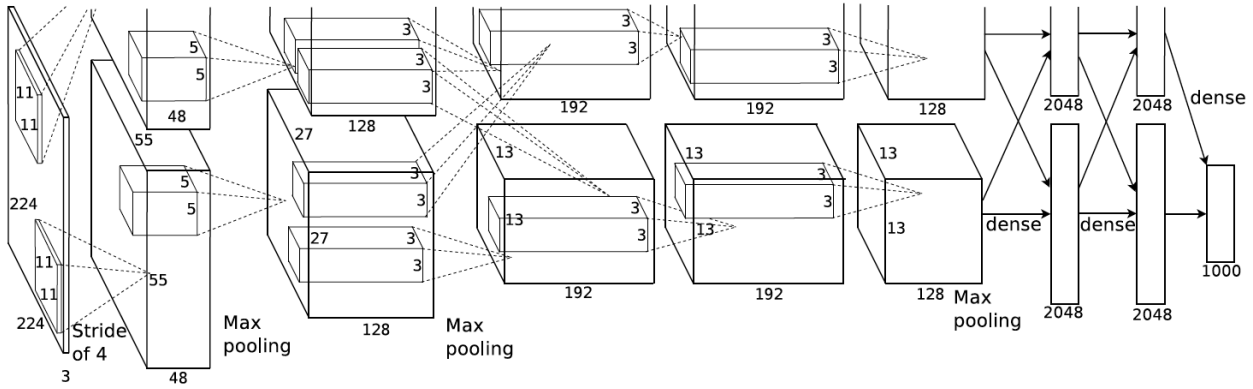
- Trained on GTX 580 with 3GB of memory.
- Network and feature maps are spread across two GPUs.
- Cross-talk across certain layers.





Architecture Design

AlexNet [Krizhevsky et. al, 2012]



Input 224× 224 ×3

1. CONV: K=96×11×11, S=4, P=0
2. MAXPOOL: K=3×3, S=2
3. NORM: Normalisation Layer (No longer used)
4. CONV: K=256× 5 × 5, S=1, P=2
5. MAXPOOL: K=3×3, S=2
6. NORM: Normalisation Layer (No longer used)
7. CONV: K=384× 3 × 3, S=1, P=1
8. CONV: K=384× 3 × 3, S=1, P=1
9. CONV: K=256× 3 × 3, S=1, P=1
10. MAXPOOL: K=3×3, S=2
11. FC: 4096D
12. FC: 4096D
13. FC: 1000 (number of classes in ImageNet)

Local Response Normalisation Layers:

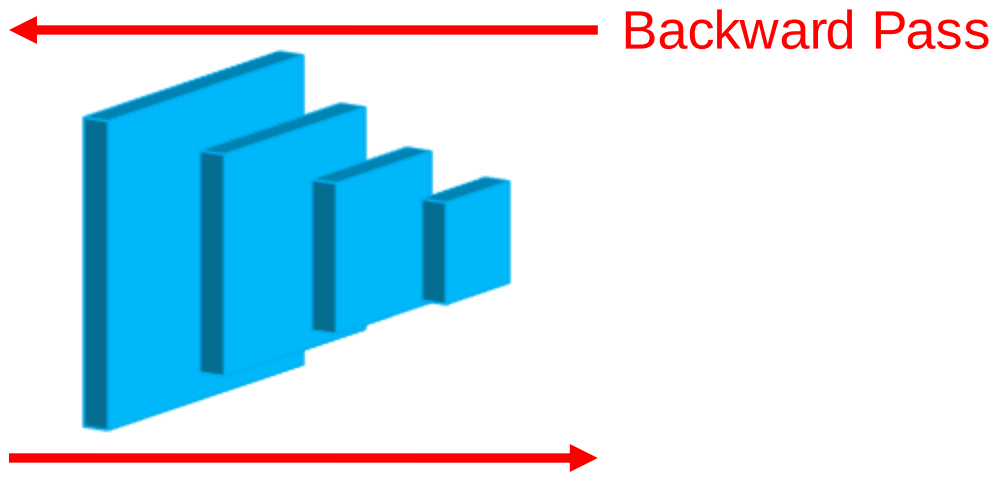
Normalising the response of the ReLU across neighbouring channels.
(Demonstrated to be ineffective)

Batch Normalisation is common.

Building Blocks

Batch Normalisation

All layers are updated in the backward pass.
Every layer assumes other layers are fixed.



The weights of a layer can be updated based on the expectation that the prior layer produces values with a given distribution. However, this distribution is likely to change once that layer is updated.

Problem: Internal Covariate Shift

(change in the distribution of input during training)

Solution: Batch Normalisation

(scaling the output of every layer to a standard distribution)

Building Blocks

Batch Normalisation

Layer outputs are rescaled such that they have a mean of *zero* and a standard deviation of *one*, *i.e.* a **standard Gaussian**.

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mathrm{E}[x^{(k)}]}{\sqrt{\mathrm{Var}[x^{(k)}]}}$$

The layer is then allowed to learn two new parameters (γ and β), using which identity mapping can be recovered if the learning process needs it.

$$\hat{x}^{(k)} = \gamma^{(k)}\ \hat{x}^{(k)} + \beta^{(k)}$$

At test time, a running mean and variance calculated during training is used for normalisation.

Building Blocks

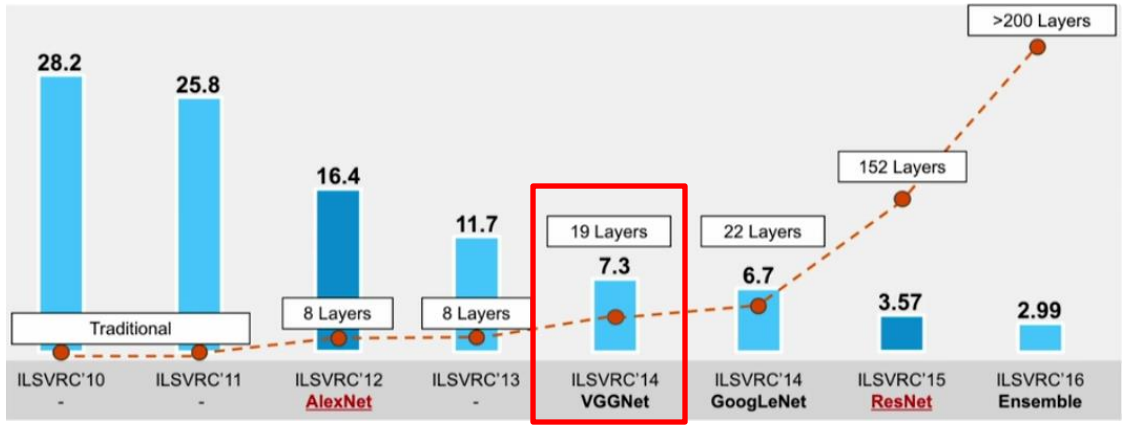
Types of Normalisation

There are various other normalisation methods (for further reading):

- Weight Norm: separates the norm of the weight vector from its direction.
- Layer Norm: normalises the inputs across the features (instead of batches).
- Instance Norm: normalises the inputs across each channel.
- Group Norm: normalises over group of channels for each training examples.
- Batch-Instance Normalisation
- Switchable Normalisation
- … .

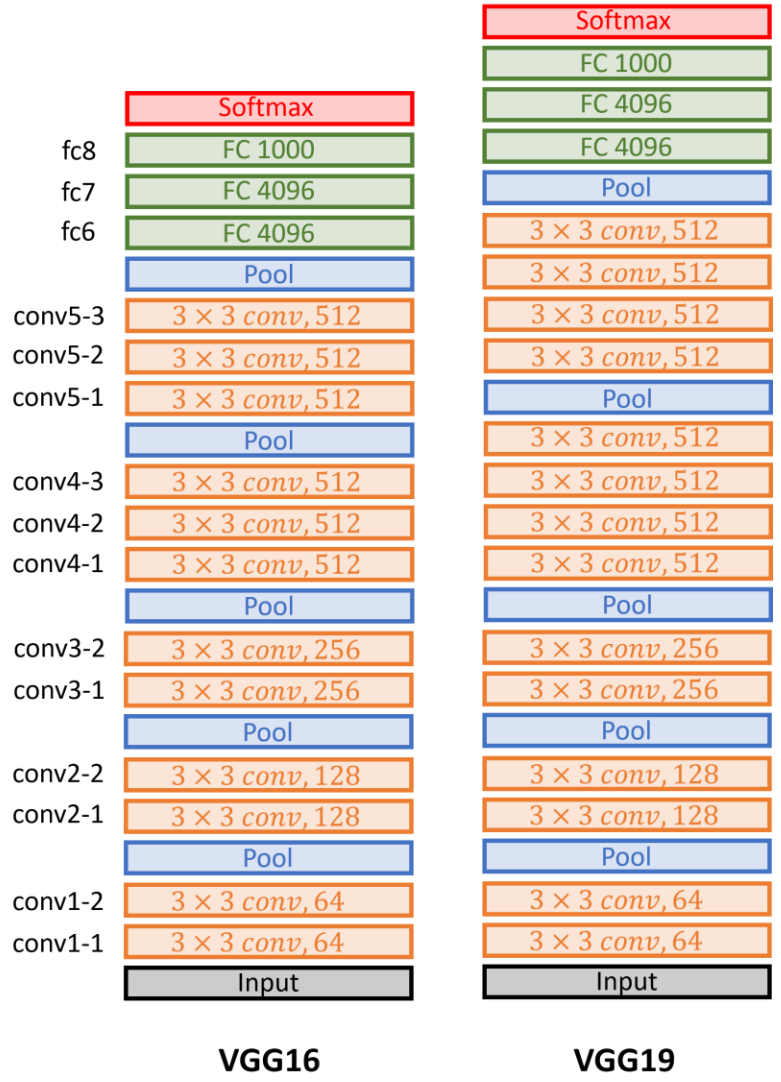
Architecture Design

VGG [Simonyan and Zisserman, 2014]



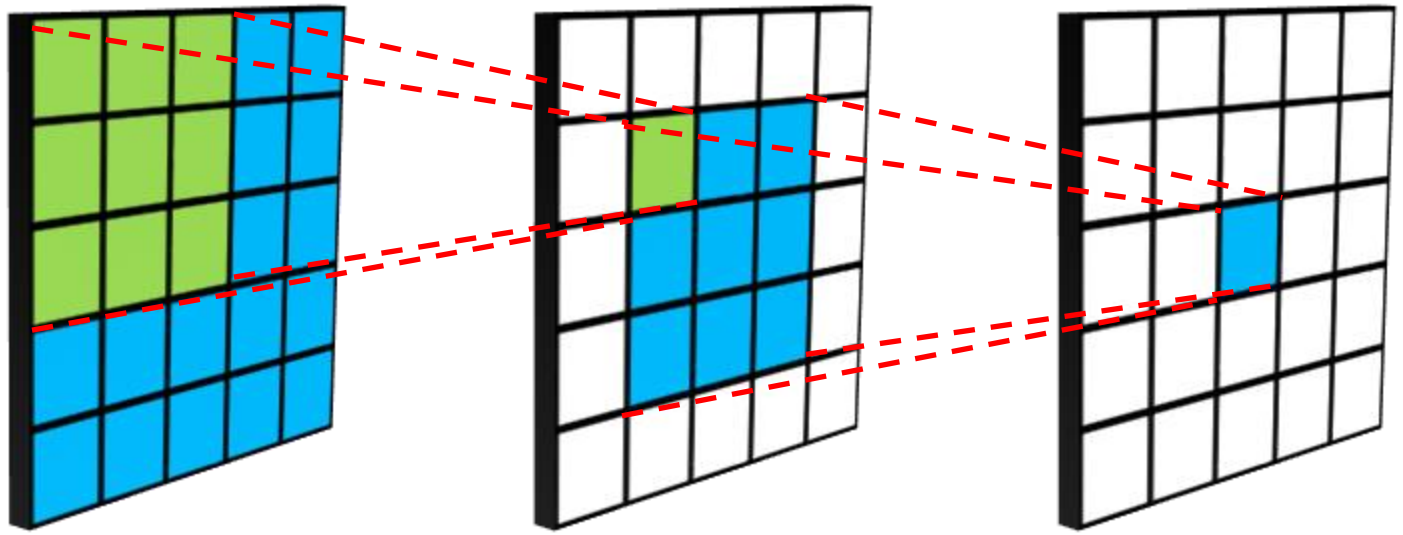
Second place in the ImageNet Large Scale Visual Recognition Challenge in 2014.

- Deeper: more layers (16 and 19)
- Smaller kernels: stacks of 3×3 convolutions
  - Fewer parameters
  - Same effective receptive field



Receptive Field

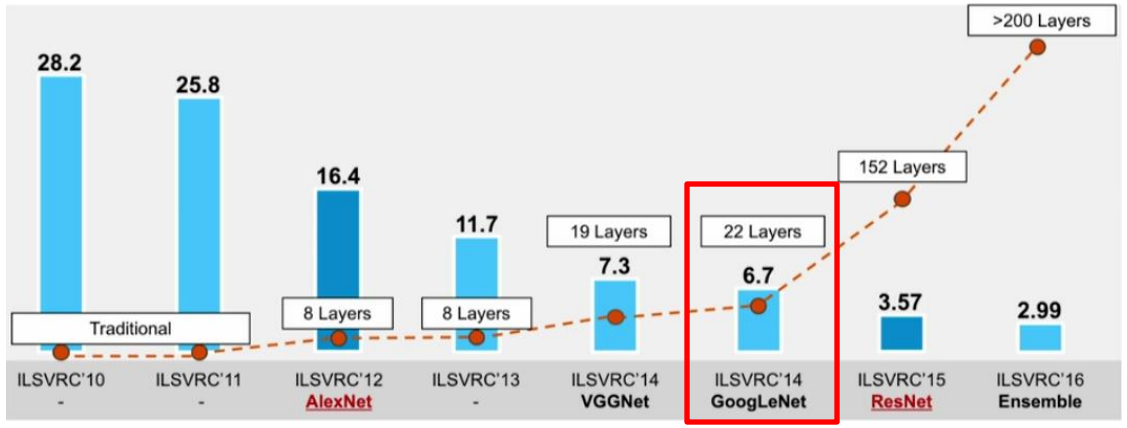
- **Biology**: portion of sensory space that can elicit neuronal responses, when stimulated.
- **Deep learning**: size of the region in the input that produces the feature at any given layer.



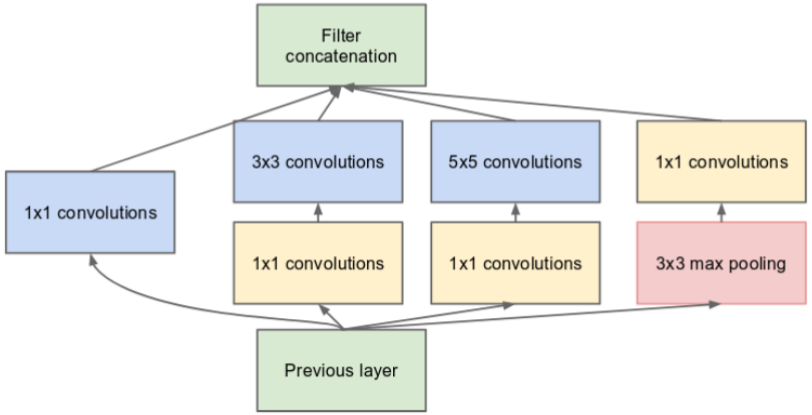
Decision-making neurons should be able to see the entire input.

Architecture Design

GoogLeNet (Inception) [Szegedy et al., 2014]



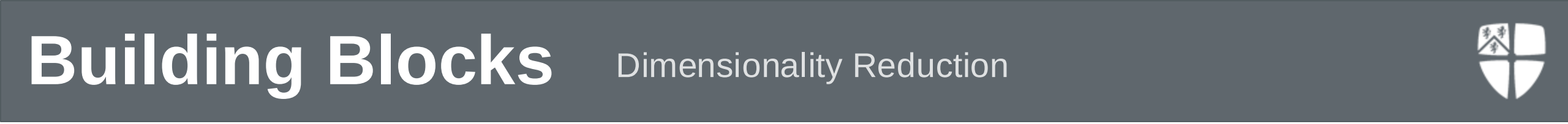
Winner of the ImageNet Large Scale Visual Recognition Challenge in 2014.



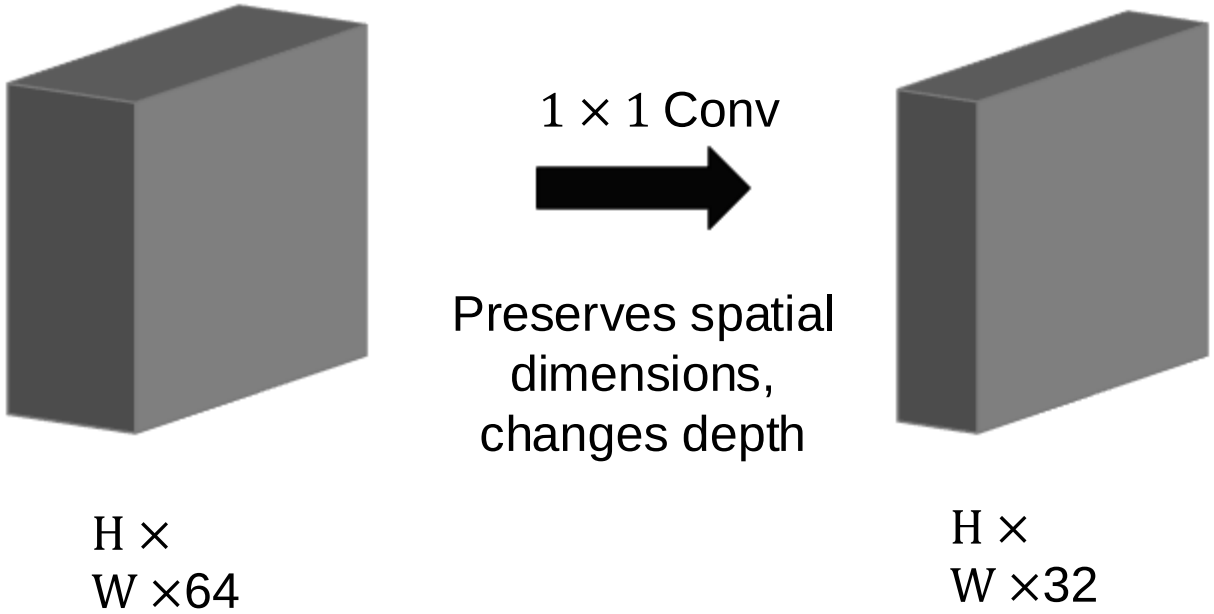
- Deeper but with fewer parameters
- No Fully-connected layers
- Uses “Inception” modules
  - Multiple kernel operations in parallel

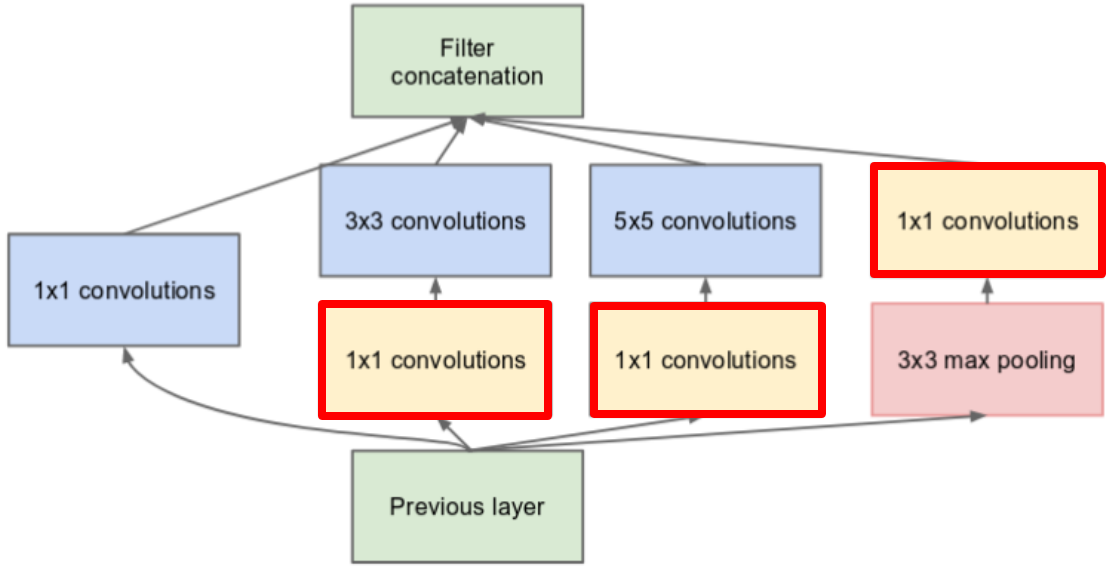
Building Blocks

Dimensionality Reduction



- Different convolution operations in parallel enable capturing localities and structures of different sizes in the image.
- 1×1 Convolutions can reduce feature dimensionality to better control the depth of features before they are concatenated in the output.



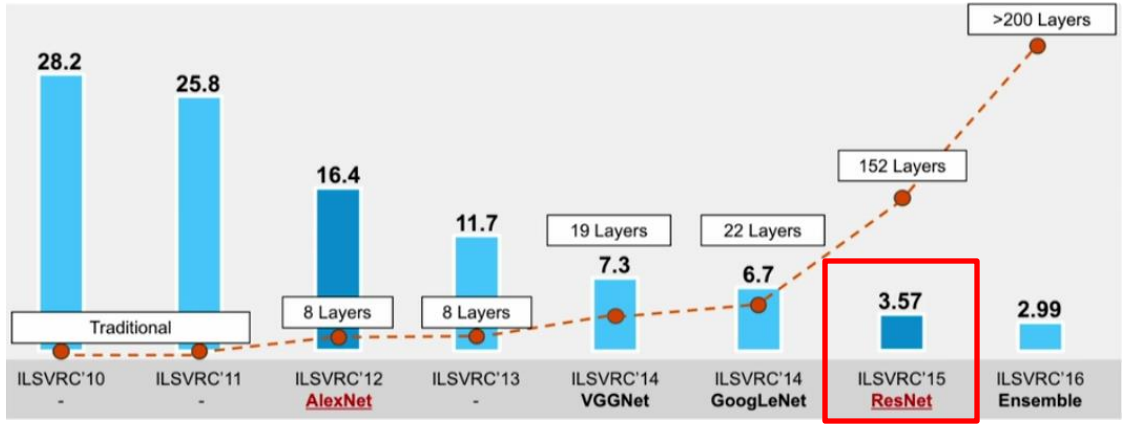


Architecture Design

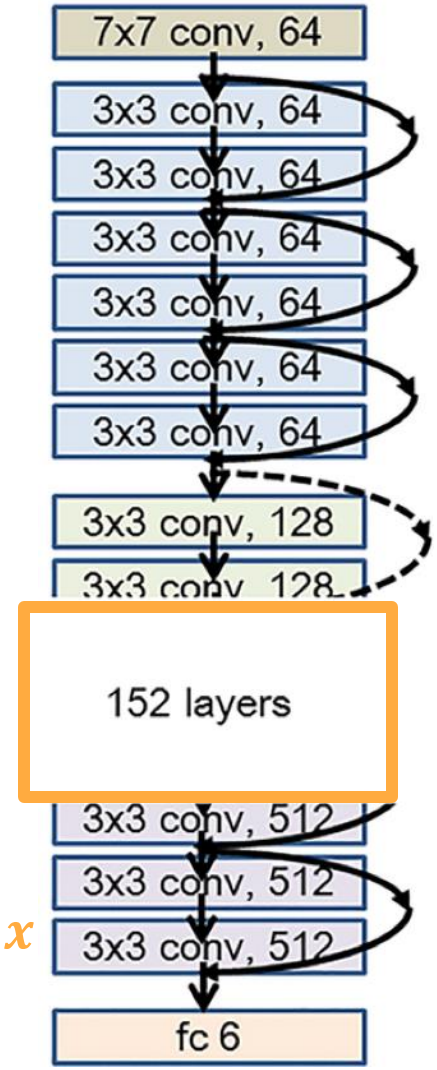
ResNet [He et al., 2015]

Very deep.

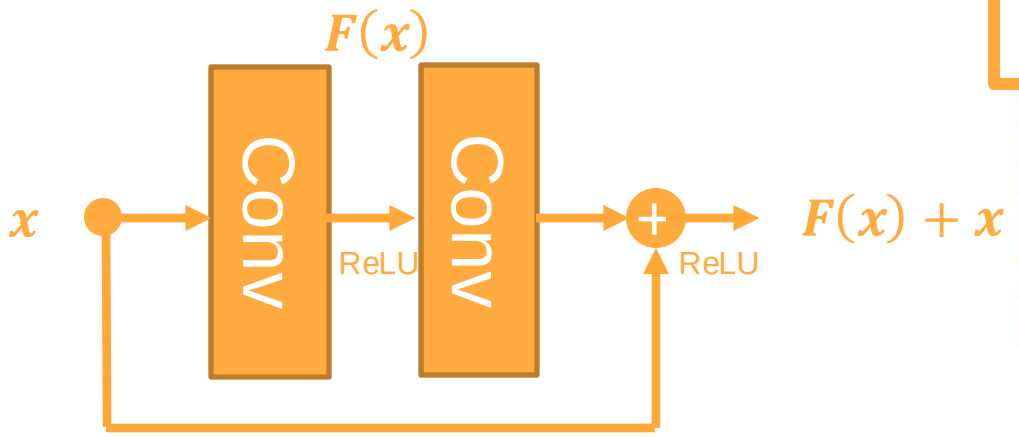
- Up to 152 layers.
- Optimisation is difficult.



Winner of the ImageNet Large Scale Visual Recognition Challenge in 2015.



- Residual modules are introduced to learn a residual mapping instead of a direct mapping.

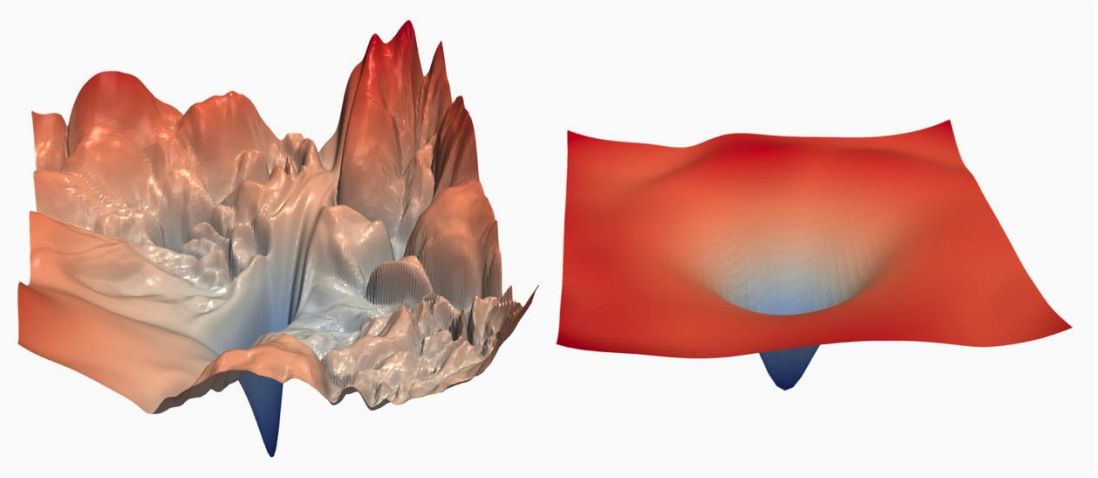


- No fully-connected layers.
- Depths of 18, 34, 50, 101, 152.

Building Blocks

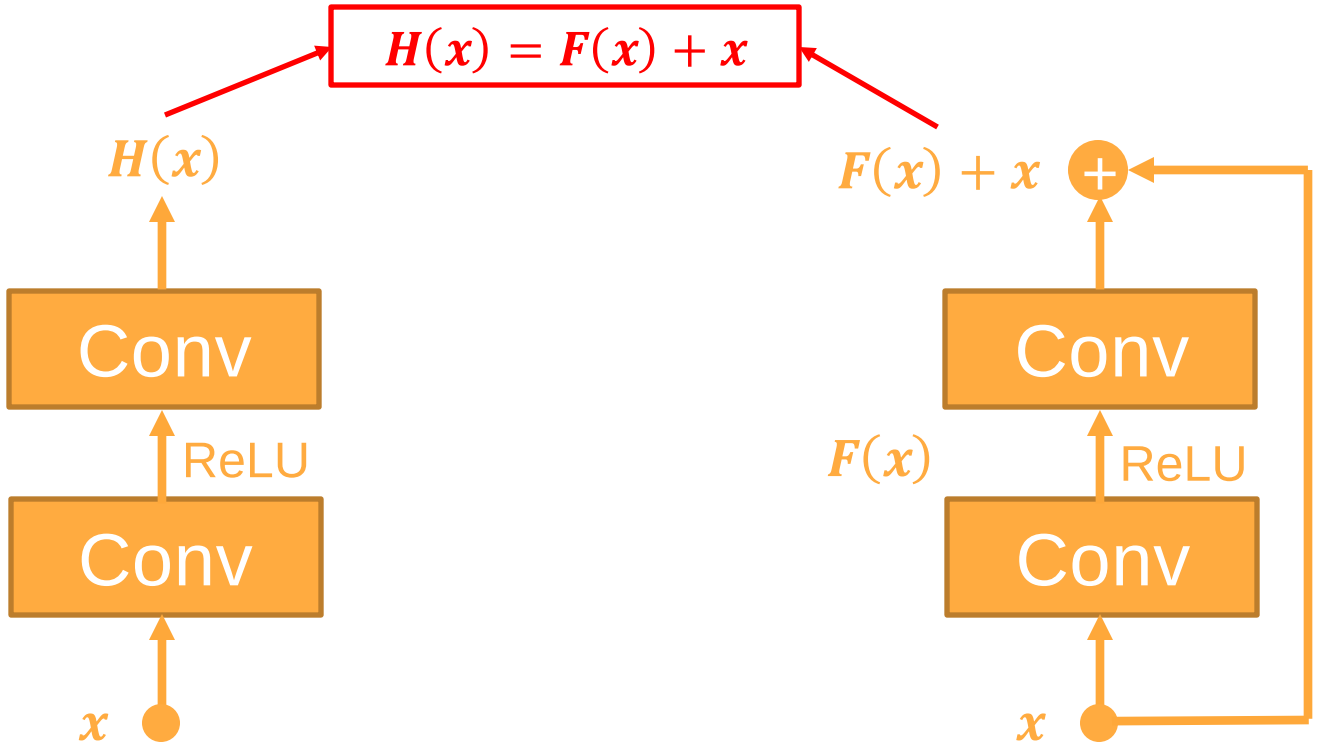
Residual Connections

- A “plain layer” learns a direct mapping from the input to the output.
- A **Residual block** learns a residual to the identity mapping.



Loss landscape without residuals

Loss landscape with residuals

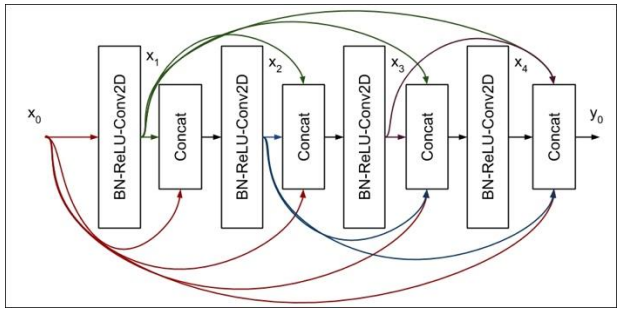


Plain Layer

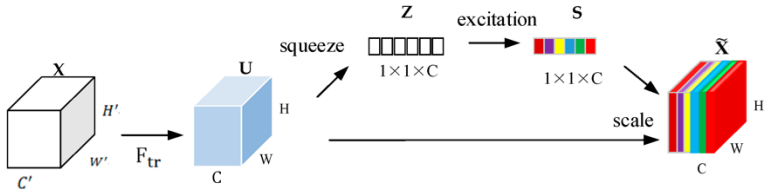
Residual Layer

Architecture Design The list is long…!

- DenseNet

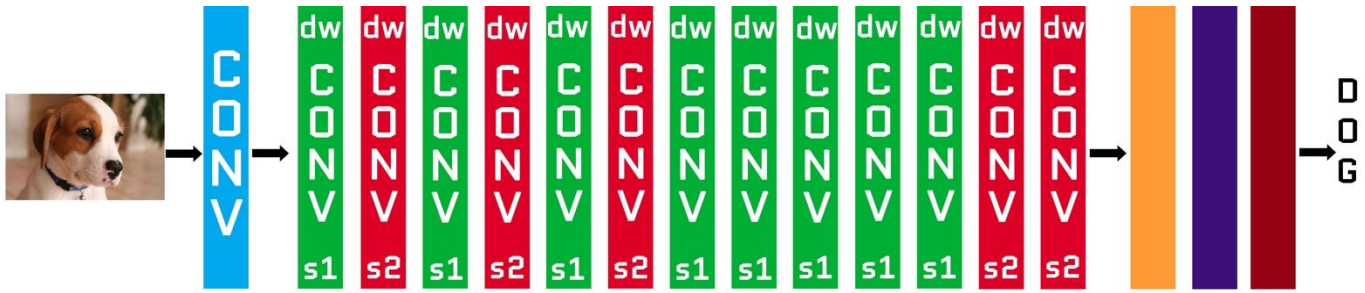


- SENet



- MobileNet

Depthwise Separable Convolutions



- ConvNeXt
- MaxVit
- MNASNet
- RegNet
- ResNeXt
- ShuffleNet
- SqueezeNet
- SwinTransformer
- VisionTransformer
- Wide ResNet
- ….

https://pytorch.org/vision/stable/models.html

Better Training

Poor Model Performance

Poor Performance is often caused by:

- Wrong architecture design
- Model Capacity
  - how complicated a pattern or relationship can the model express?
  - how complex a function can the model approximate?
- Wrong hyperparameters
- Overfitting
  - when the model fits a particular set of data too closely and does *not* generalise to future unseen data.
  - Model performs well on the training data.
  - Model is too complex OR dataset is too small.
  - Model cannot perform on the test data it has not seen yet.
  - *essentially memorising noise patterns in the training data.*

Better Training

Poor Model Performance

Overfitting (an example):

Find the next number in the sequence

1, 3, 5, 7, ?

We can use a quartic function to solve this!

$$f(x) = \frac{18111}{2}x^4 - 90555x^3 + \frac{633885}{2}x^2 - 452773x + 217331$$

$x =$ Index in sequence

$f(1) = 1$ $\quad$ $f(3) = 5$

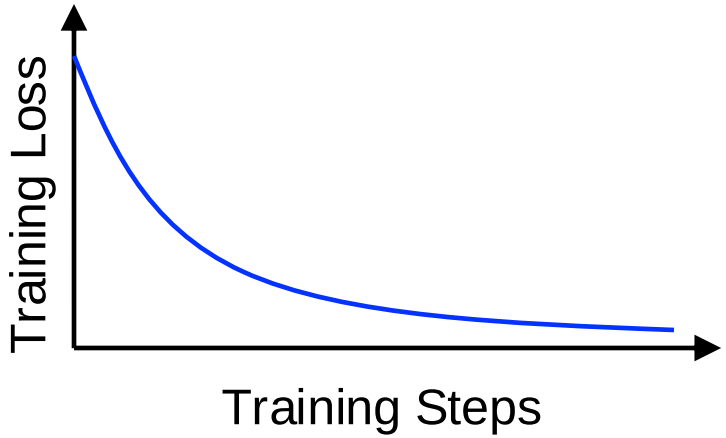
$f(2) = 3$ $\quad$ $f(4) = 7$

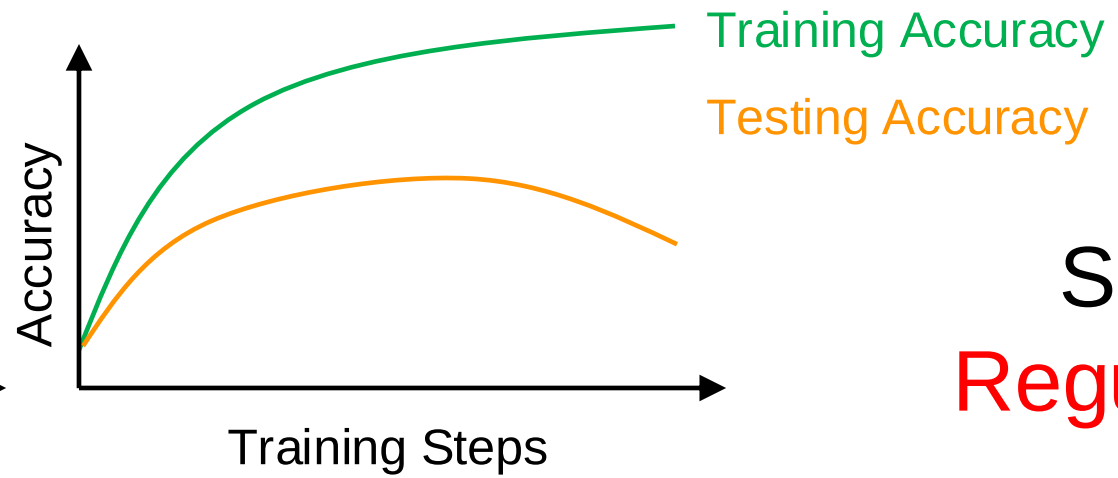
$f(5) = 217341$

Better Training

Poor Model Performance

Overfitting is very common in deep learning applications.





Solution:
Regularisation

L_2 Regularisation

$$R(W) = \sum_i \sum_j W_{ij}^2$$

L_1 Regularisation

$$R(W) = \sum_i \sum_j |W_{ij}|$$

Elastic net

$L_1 + L_2$

Better Training

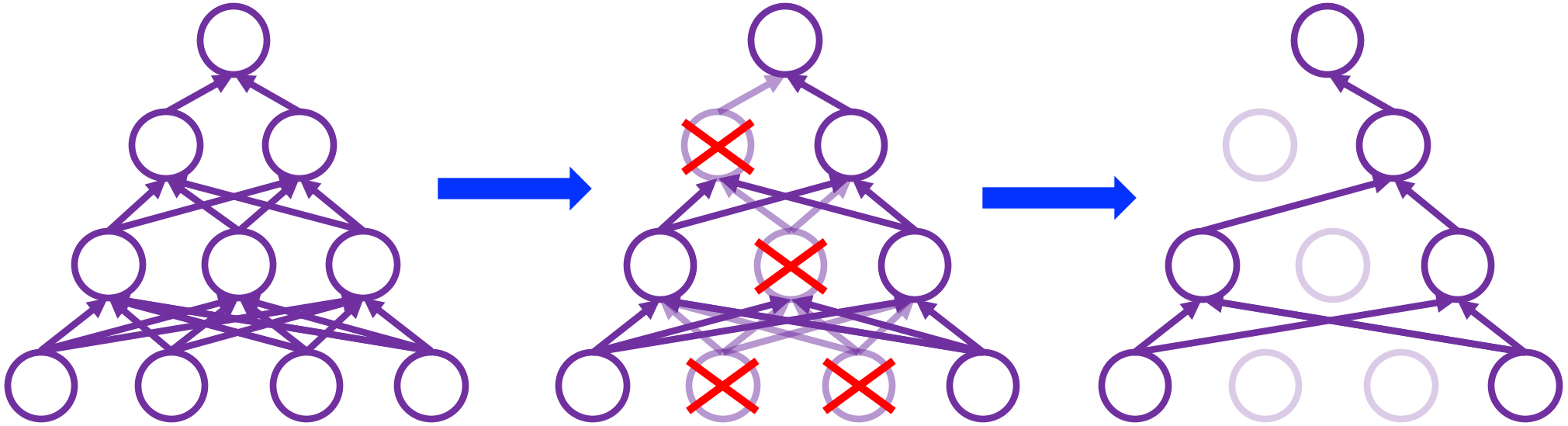
Dropout

Dropout is A regularisation strategy – *theoretically the same as L_2.*

- During each forward pass, randomly **drop out** some of the neurons.

probability is a hyperparameter

set neuron activations to zero



Better Training

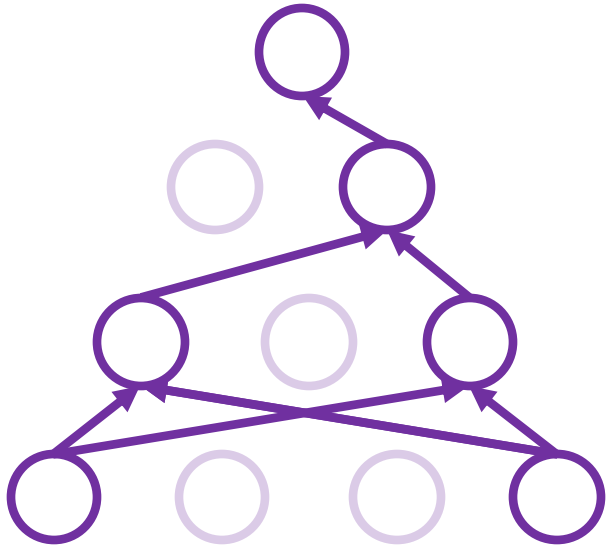
Dropout

How does it improve generalisation?

- Forces the model to learn redundant features.

  (The model will learn several different ways of reaching the same answer)

- As if we are training several different models contained within the same one (ensemble).



Better Training

Dropout

We have introduced some randomness into the training process.

$$y = f_W(x, z)$$

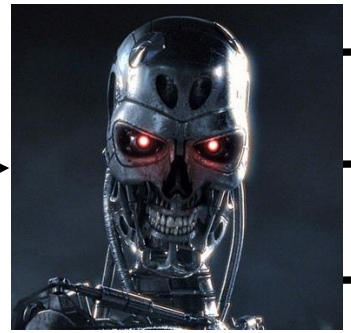
Output (class label)

Model (with weights W)

Input (image)

Randomness (dropout mask)

We really don't want randomness at all during test time.



Monday? → **Cat**

Tuesday? → **GeForce RTX 4090**

Sunday? → **John Connor**

Better Training

Dropout

We can try to average out the randomness during test time!

$$y = \mathbb{E}_z[f(x,z)] = \int p(z) f(x,z) dz$$

Interactable!!

We can simply approximate this locally and cheaply:

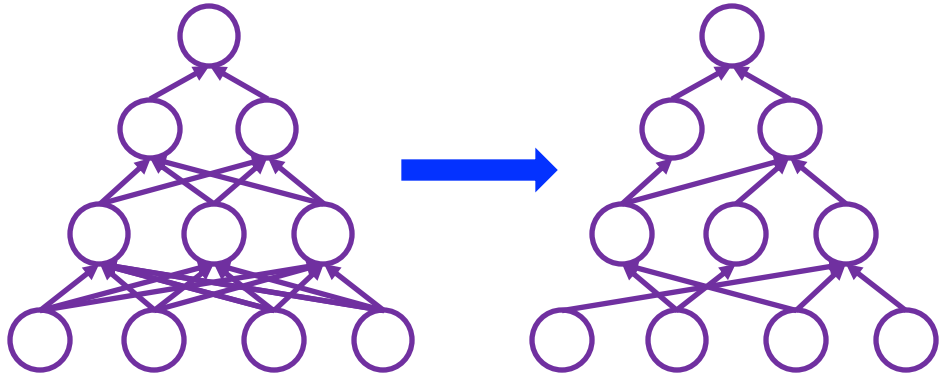
- Keep all neurons active during test time.
- Multiply the output of the neurons by the dropout probability value.

Better Training

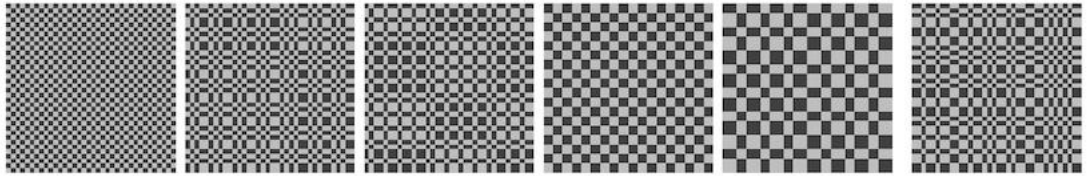
Other Ideas similar to Dropout

other ways of introducing stochasticity during training:

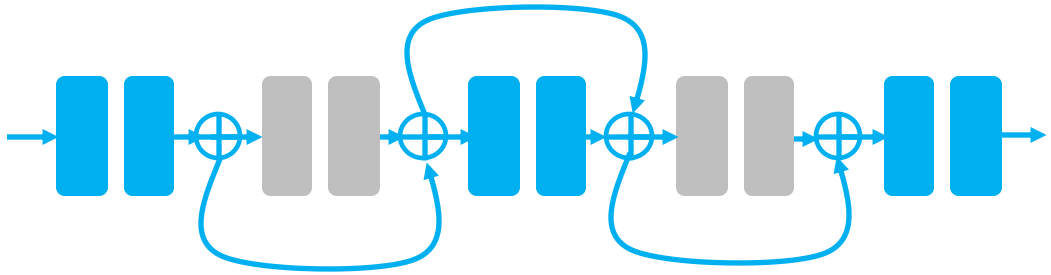
- **DropConnect:** Instead of removing neuron activations, weight values (connections) are removed.



- **Fractional Max Pooling:** Randomising the regions over which pooling happens at every forward pass.



- **Random Network Depth:** Randomly dropping entire layers during training.

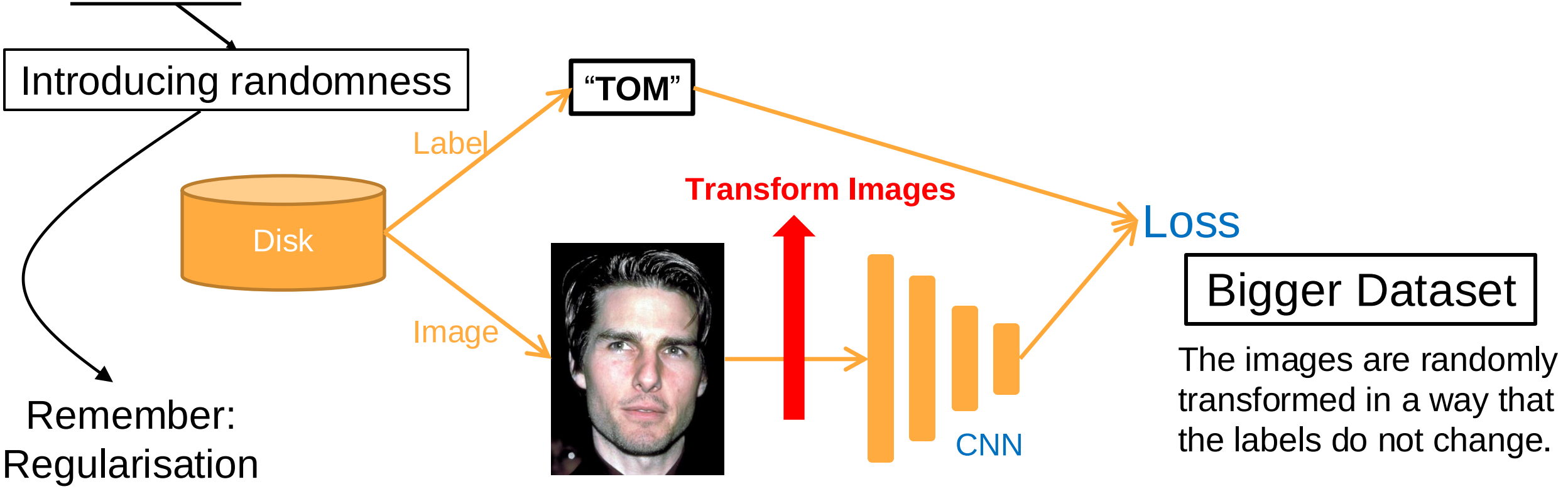


Improve Performance

Data Augmentation

One of the causes of overfitting is the small size of the dataset.

- Randomly transforming the input image to “augment” the dataset.



Improve Performance

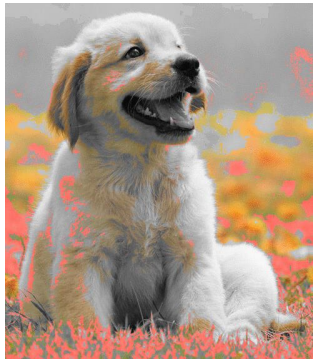
Data Augmentation

Your choice of transformations should depend on task and data.

Learned Augmentation…

Mix and Match!

Rotation



Colour Jitter

Brightness and Contrast

Random Crop and/or Scale during training

Fixed Crop and/or Scale during testing

Perspective

Blur (Motion Blur)

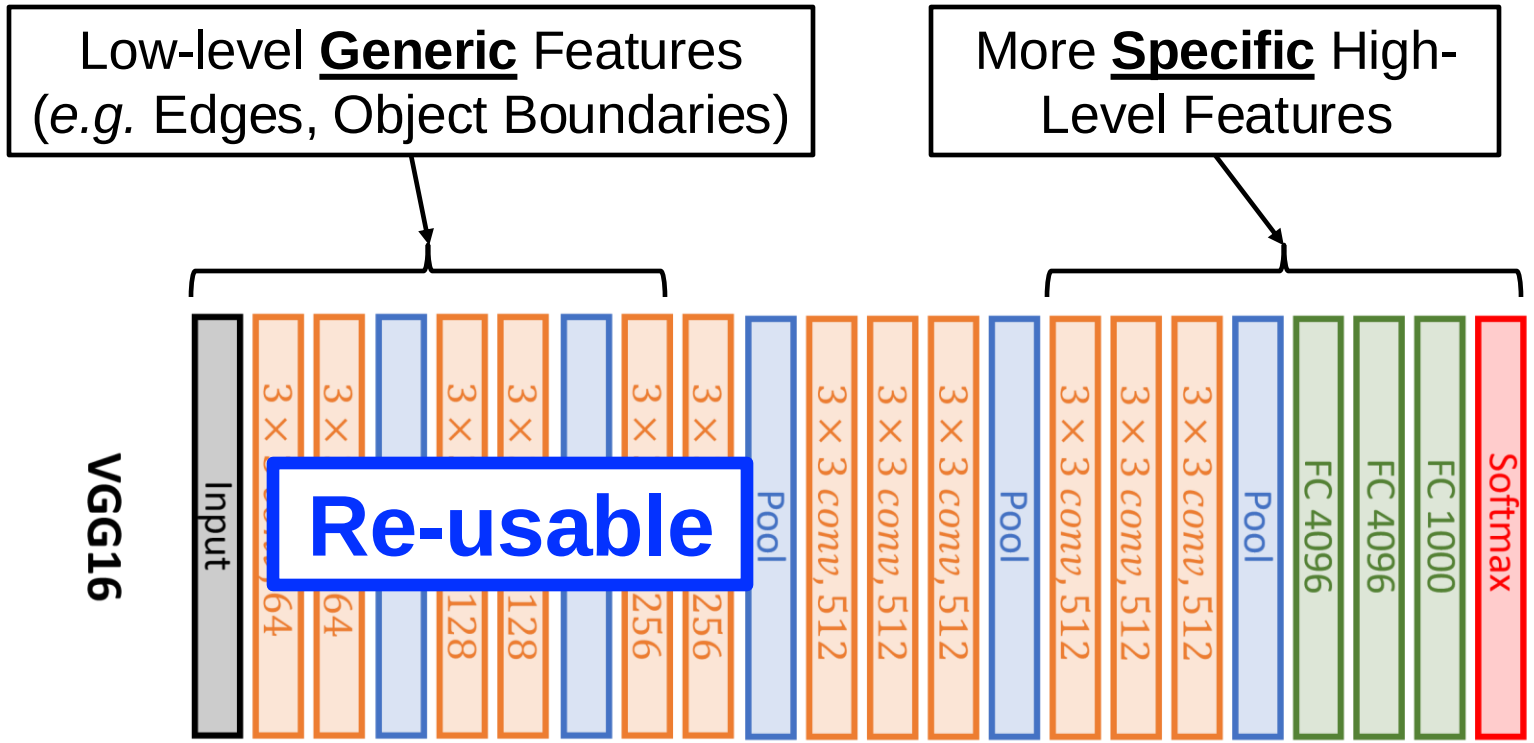
or any other blur you want!

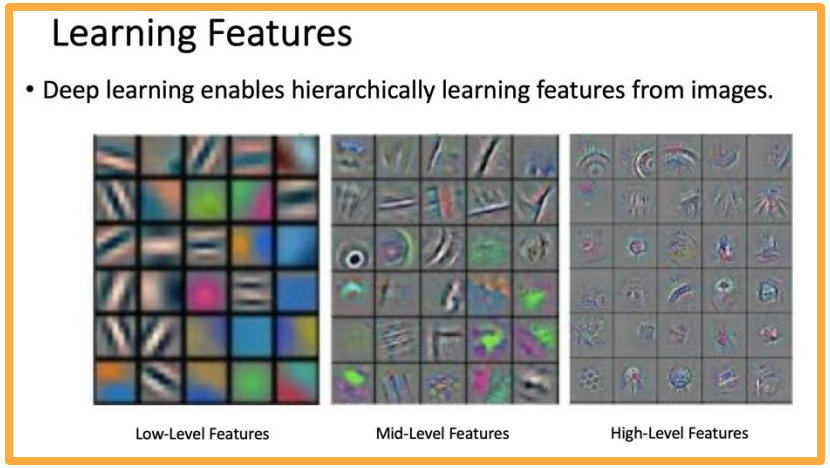
No Limit as to what can be done!

Improve Performance

Transfer Learning

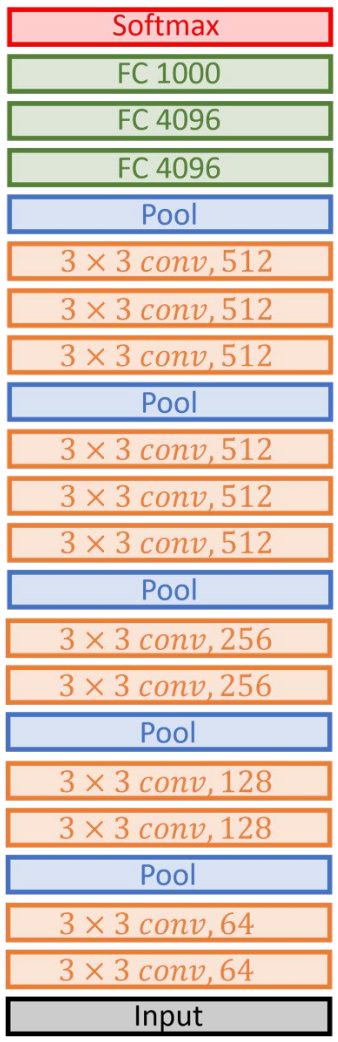
- When the training dataset is very small, there is only so much data augmentation can do.
- Remember that our model is meant to capture hierarchical features.





Improve Performance

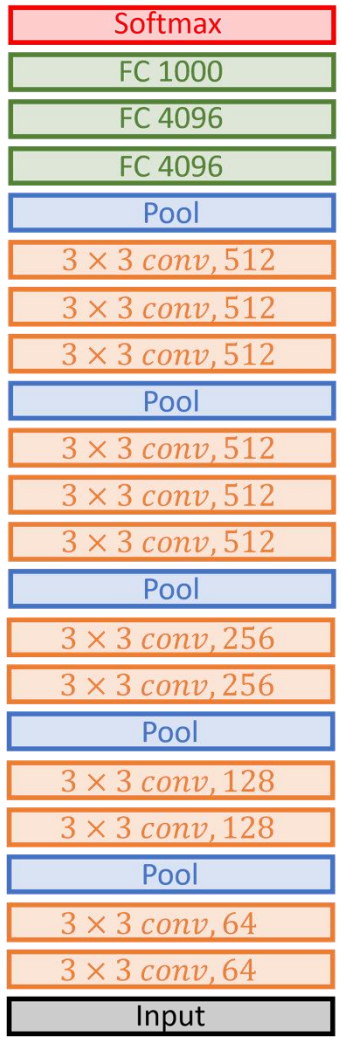
Transfer Learning



At first, we train our CNN on a **large dataset**, like ImageNet.

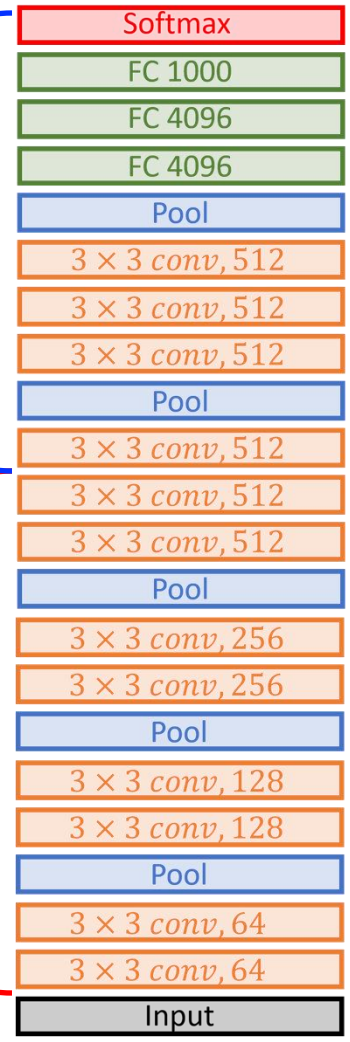
We then want to make this work for our **very small dataset**.

The larger our dataset, the more of the network we fine-tune (maybe all of it).



Reinitialise & train higher layers (e.g. classifier). This is called “Fine-tuning”

“Freeze” layers with Generic features.



Improve Performance

Transfer Learning

- When fine-tuning, keep the learning rate smaller.
- The number and position of layers that are frozen and fine-tuned depends on the dataset (can be thought of as a hyper-parameter).

Using CNNs Pre-trained on ImageNet is extremely common and can be very powerful.

You are not allowed to use any transfer learning for your coursework!

size of our dataset and its similarity to the pre-training data

	Similar data	Different data
Little data	only fine-tune the classifier at the top	try cutting out some top layers completely? BAD
A lot of data	fine-tune more layers from the top	fine-tune a large number of top layers

Let’s Look at Some Code!

Better Training Example

Code on GitHub : https://github.com/atapour/dl-pytorch/blob/main/Better_Training/Better_Training.ipynb

Code on Colab: https://colab.research.google.com/github/atapour/dl-pytorch/blob/main/Better_Training/Better_Training.ipynb

What we learned today!

Summary

Designing architectures:

- is a scientific process
- design the right shape to transform the data
- choose the right functions to fit the data
- evaluate your models carefully
- be careful about overfitting
- what does the task need?